

# Recap

→ A buggy code where total = price should be total = total + price

```
total = 0
for price in prices:
    total = price
    print(total)
```

→ Tokenization — →

→ Vectorization —  
Each id becomes a vector capturing meaning

→ Positional Encoding → To the generated vector we added what's the position of each word in sentence.

→ Attention → Each word looks around itself and absorbs the context

→ Feed-Forward NN ( Pattern detectors which make the meanings in the vector further clear.

x Model now understand's the code?  
The bug

Now the model has to answer back to user

---

The **last vector** in this sequence — the one sitting at the position of "?" — is the one that matters most.

This vector encodes everything the model understood

— the code,  
the bug,  
your question about it.

How has the initial vector of ? evolved

*Early layers: "? is a question mark at the end of a sentence" -> [0.1, 0.5, 1.9]*

*Middle layers: "the user asked a question about a Python bug" -> [0.6, 0.7, 1.9]*

*Deep layers: "the answer should explain the accumulator bug on line 3" -> [0.9, 0.6, 1.9]*

Remember back at the start, there was an **embedding table** — a lookup that converts each token ID to a vector?

Now the model does **the reverse**. It compares  $v_{\text{final}}$  against every *single entry* in that same embedding table.

The comparison is a dot product because it measures similarity

The 50,000 numbers that come out of the comparison are called **logits**. They're raw similarity scores.

It picks the best match.

Append and repeat.

? → "The"

? The → "bug"

? The bug → "is"

? The bug is → "on"

? The bug is on → "line"

? The bug is on line → "3"

## The complete picture, from FFN output to next token

Last FFN layer produces final vectors



Take the LAST vector ( $v_{\text{final}}$ )



Compare  $v_{\text{final}}$  against EVERY word in the embedding table  
(dot product: "how similar is my understanding to each word?")



50,000 raw scores ← these are called LOGITS



Pick one word (greedy or sampling)



Append to input, run the whole pipeline again



Repeat until done

# LLM Model

LLM = cut off  
GPT 5.5  
JAN 2024

Training

millions of  
Data on  
internet

Inference

I'm trained now  
I can answer  
the user questions.

Learn Relationship

What happened During the training time.

| x | y |
|---|---|
| 1 | 3 |
| 2 | 5 |
| 3 | 7 |
| 5 | ? |

$y = ax + b$      $a = 4$      $b$   
it learnt the value of a  
and b. (the relationship)

Internet (Self supervised Learning)

The cat sat on the ~~xxx~~ mat.

The cat sat on the ~~xxx~~.

Predict the next word.

Model will initially predict wrong

The cat sat on the table ✗

I already know the answer

The cat sat on the mat

mat  $\neq$  table ✗

Calculate the Loss

Remember our

$$y = ax + b$$

$$\begin{aligned} a &= 1 \\ b &= 1 \\ y &= ax + b \end{aligned}$$

| $x$ | $y$ | $y'$ |                                        |
|-----|-----|------|----------------------------------------|
| 1   | 3   | 2    | } Loss = 1<br>} Loss = 1<br>} Loss = 2 |
| 2   | 5   | 4    |                                        |
| 3   | 7   | 5    |                                        |

$$\begin{aligned} a &= ? \\ b &= ? \end{aligned}$$

Go and update a and b  
based on loss. It will

do the something go back  
and update the model.

# Training

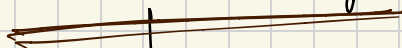
Input



tokenization



Embeddings



positional  
Encoding



Attention



Feed Forward  
NN



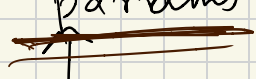
Predict

Right (Do Nothing)

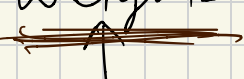
[ Embedding  
table ]



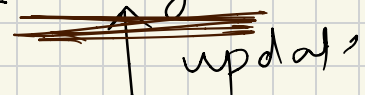
[ position  
related  
params ]



[ Attention  
weights ]

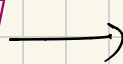


[ FFN  
weights ]



update

Wrong



| <u>Model</u> | <u>parameters</u> | $y = ax + \frac{b}{x}$ |
|--------------|-------------------|------------------------|
| <u>gpt-2</u> | 1.5 million       | 3 GB file              |
| <u>gpt-3</u> | 175 billion       | 350 GB file            |
| <u>gpt-4</u> | 1.8 trillion      | 2 TB file              |

## Training Data      Doc Size

|              |                |                   |
|--------------|----------------|-------------------|
| <u>gpt 2</u> | 40 GB of text  | → few days on GPU |
| <u>gpt 3</u> | 570 GB of text | → weeks           |
| <u>gpt 4</u> | 10 TB of text  | → month           |

## Attention (QKV Matrices)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Where:

- **Q** (query), **K** (key), and **V** (value) are all projections of the input tokens.
- $d_k$  is the dimensionality of the keys.
- The dot product  $QK^T$  measures similarity between tokens.
- Softmax ensures the attention weights sum to 1.

To understand the word "sat," self-attention computes its relationship to every word in the sentence, including itself. The model learns, for example,

## Attention, in plain English:

- 1 Every word starts knowing only its dictionary meaning
- 2 Each word looks back at the words before it and asks: "which of you matter to me?"
- 3 It then pulls in a little bit of meaning from each one — more from the important ones, less from the rest.
- 4 Now that word "knows" not just itself, but the story around it.
- 5 The **last word ends up holding the meaning of the whole sentence** — And that's what the model uses to guess the next word.

The cat sat on the Mat

After embedding + positional encoding, we have 3 vectors.

Let's use **tiny 4-dimensional vectors** so we can actually do the math by hand:

The → [1, 0, 1, 0]  
cat → [0, 2, 0, 2]  
sat → [1, 1, 1, 1]

(In a real LLM these would be 4096 numbers long, but the idea is identical.)  
These vectors represent each word's **dictionary meaning + position**.



Every word plays three roles during attention:

- 1 **Asks** a question ("who matters to me?") - Query
- 2 **Advertises** itself ("here's what I am") - Key
- 3 **Hands over** its meaning ("here's my content to take")- Value

We **transform** each word's vector into three different versions — one for each job.

The Cat Sat on Mat

**cat - Q , K , V**

**Query** - > *What action am I doing? Where am I?*

**Key** -> *I'm a living animal. I perform actions*

**Value** -> *small furry pet, four-legged mammal, independent creature, mouse-chaser, household companion.*

---

How do we transform a vector? **By multiplying it with a matrix.**

- **W\_Q** — the matrix that turns any word into its "Query" version
- **W\_K** — the matrix that turns any word into its "Key" version
- **W\_V** — the matrix that turns any word into its "Value" version

These three matrices are **the same for every word.**

Let's pretend these matrices look like this (4×4 each, made-up small numbers):

$$\begin{array}{lll} W\_Q = \begin{bmatrix} 1, & 0, & 1, & 0 \\ 0, & 1, & 0, & 1 \\ 1, & 0, & 0, & 1 \\ 0, & 1, & 1, & 0 \end{bmatrix} & W\_K = \begin{bmatrix} 0, & 1, & 0, & 1 \\ 1, & 0, & 1, & 0 \\ 0, & 0, & 1, & 1 \\ 1, & 1, & 0, & 0 \end{bmatrix} & W\_V = \begin{bmatrix} 1, & 1, & 0, & 0 \\ 0, & 1, & 1, & 0 \\ 0, & 0, & 1, & 1 \\ 1, & 0, & 0, & 1 \end{bmatrix} \end{array}$$

Don't worry about where these numbers come from — they're learned during training. Just treat them as given for now.

### Step 3: Generate Q, K, V for Every Word

For each word, we multiply its vector by all three matrices.

#### For "cat" (vector = [0, 2, 0, 2])

$[0, 2, 0, 2] \times W\_Q = [0, 4, 2, 2]$  Q\_cat  
-> "cat"'s question -> *What action am I doing?*

$[0, 2, 0, 2] \times W\_K = [4, 2, 2, 0]$  K\_cat =  
-> "cat"'s label -> *I'm a living animal.*

$[0, 2, 0, 2] \times W\_V = [2, 2, 2, 0]$  V\_cat  
-> "cat"'s actual content -> *I carry: small furry pet, four-legged mammal*

**For "sat" (vector = [1, 1, 1, 1])**

$[1, 1, 1, 1] \times W_Q = [2, 2, 2, 2]$  Q\_sat

"sat"'s question -> *Who did this sitting?*

$[1, 1, 1, 1] \times W_K = [2, 2, 2, 2]$  K\_sat =

"sat"'s label -> *I usually need a subject (who) and often a location (where).*

$[1, 1, 1, 1] \times W_V = [2, 2, 2, 2]$  V\_sat

← "sat"'s actual content -*the act of lowering one's body onto a surface*

## What Just Happened

Before this step:

The → [1, 0, 1, 0]

cat → [0, 2, 0, 2]

sat → [1, 1, 1, 1]

After this step:

|          | Q (its question) | K (its label) | V (its       |
|----------|------------------|---------------|--------------|
| content) |                  |               |              |
| The →    | [2, 0, 1, 1]     | [0, 1, 1, 2]  | [1, 1, 1, 1] |
| cat →    | [0, 4, 2, 2]     | [4, 2, 2, 0]  | [2, 2, 2, 0] |
| sat →    | [2, 2, 2, 2]     | [2, 2, 2, 2]  | [2, 2, 2, 2] |

## Now the Word "sat" Does Attention (Look around)

### "sat" asks its question

"sat" picks up its Query:  $Q\_sat = [2, 2, 2, 2]$  (This is "sat" saying: *"Who did this sitting?"* )

### Compare sat's Query against every word's Key

Comparison is done via **dot product** — that's just "multiply each pair of numbers, then add them all up." It measures how aligned two vectors are. High number = aligned = relevant.

$$\begin{aligned} Q\_sat \cdot K\_The &= [2, 2, 2, 2] \cdot [0, 1, 1, 2] \\ &= (2 \times 0) + (2 \times 1) + (2 \times 1) + (2 \times 2) \\ &= 0 + 2 + 2 + 4 \\ &= 8 \end{aligned}$$

$$\begin{aligned} Q\_sat \cdot K\_cat &= [2, 2, 2, 2] \cdot [4, 2, 2, 0] \\ &= (2 \times 4) + (2 \times 2) + (2 \times 2) + (2 \times 0) \\ &= 8 + 4 + 4 + 0 \\ &= 16 \end{aligned}$$

$$\begin{aligned} Q\_sat \cdot K\_sat &= [2, 2, 2, 2] \cdot [2, 2, 2, 2] \\ &= (2 \times 2) + (2 \times 2) + (2 \times 2) + (2 \times 2) \\ &= 4 + 4 + 4 + 4 \\ &= 16 \end{aligned}$$

So far we have raw relevance scores:

The: 8                  cat: 16                  sat: 16

"cat" and "sat" both score high. "The" scores lower.

## Scale the scores

Divide by  $\sqrt{(\text{dimension})} = \sqrt{4} = 2$ . This is just to keep the numbers from blowing up.

The: 4                  cat: 8                  sat: 8

## Softmax — turn scores into proper weights that sum to 1

$$e^4 = 54.6$$

$$e^8 = 2981.0$$

$$e^8 = 2981.0$$

$$\text{Sum} = 54.6 + 2981 + 2981 = 6016.6$$

$$\text{The: } 54.6 / 6016.6 = 0.009 \rightarrow \text{about } 1\%$$

$$\text{cat: } 2981.0 / 6016.6 = 0.495 \rightarrow \text{about } \underline{49\%}$$

$$\text{sat: } 2981.0 / 6016.6 = 0.495 \rightarrow \text{about } \underline{49\%}$$

These are the **attention weights** for "sat":

The: 1%                  cat: 49%                  sat: 49%

This tells us:

"cat is very relevant to sat.

The is barely relevant.

sat itself is keeping half of its own information."

**Pull in the Values, (I know the importance of each word , then me change my vector accordingly)**

$$\begin{aligned}\text{new\_sat} &= 0.01 \times V_{\text{The}} + 0.49 \times V_{\text{cat}} + 0.49 \times V_{\text{sat}} \\ &= 0.01 \times [1, 1, 1, 1] + 0.49 \times [2, 2, 2, 0] + 0.49 \times [2, 2, 2, 2] \\ &= [0.01, 0.01, 0.01, 0.01] + [0.98, 0.98, 0.98, 0.00] + [0.98, 0.98, 0.98, 0.98] \\ &= [1.97, 1.97, 1.97, 0.99]\end{aligned}$$

**That's the new vector for "sat" after attention.**

The old "sat" vector was  $[1, 1, 1, 1]$ . The new one is  $[1.97, 1.97, 1.97, 0.99]$ .

The vector got nudged — it moved toward "cat"'s meaning because "cat" had the highest weight.

"sat" now carries the meaning *"the sitting being done by the cat."*

**This Happens for Every Word, in Parallel**

"The" does attention:

"cat" does attention:

"sat" does attention: (what we just did)

All three computations happen at the same time. We get back three new enriched vectors:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

Where:

- **Q** (query), **K** (key), and **V** (value) are all projections of the input tokens.
- $d_k$  is the dimensionality of the keys.
- The dot product  $QK^T$  measures similarity between tokens.
- Softmax ensures the attention weights sum to 1.

To understand the word "sat," self-attention computes its relationship to every word in the sentence, including itself. The model learns, for example,